گزارش تطبیق FinRAGBench V برای اسناد مالی فارسی

پیاده‌سازی و مقایسهٔ 25BM و 2ColQwen و Hybrid

در این پروژه، مسیر بازیابی اسناد مالی FinRAGBench-V را برای گزارش‌های فارسی دیجی‌پی آماده کردیم. دو PDF با مجموع ۱۲۲ صفحه به تصاویر صفحه، corpus در قالب Parquet و metadata تبدیل شدند. سپس بازیابی تصویری با 2ColQwen و MaxSim، بازیابی متنی با 25BM و بازیابی Hybrid با مرتب‌سازی مجدد نامزدها پیاده‌سازی و روی یک مجموعهٔ مشترک ده‌سؤالی ارزیابی شدند.

در مقایسهٔ نهایی با 25 BM،k=10 و 2ColQwen هر دو 10@Recall برابر ۹۰٪ گرفتند. 10@MRR آن‌ها به‌ترتیب ۰٫۷۵ و ۰٫۶۱۴۴۴۴ بود. Hybrid صفحهٔ مرجع همهٔ سؤال‌ها را در ده نتیجهٔ اول پیدا کرد و با 10@MRR برابر ۰٫۹۵ بهترین نتیجهٔ این آزمایش را داشت. این نتیجه به مجموعهٔ توسعهٔ فعلی مربوط است که فقط دو صفحهٔ مرجع یکتا دارد؛ ارزیابی گستردهٔ فارسی هنوز مرحله ای جداگانه است.

هدف و دامنهٔ کار

هدف اصلی، ساخت یک مسیر محلی و قابل بررسی برای یافتن صفحهٔ مرتبط در اسناد مالی فارسی بود. حفظ نمودارها و جدول‌ها، اتصال هر نتیجه به PDF و تصویر اصلی، استفادهٔ دوباره از embedding موجود و مقایسهٔ سه روش با برچسب و معیار یکسان، اهداف اجرایی این مرحله بودند. وزن‌های 2ColQwen در این پروژه آموزش مجدد یا fine-tune نشدند.

این گزارش آماده‌سازی داده، کدهای اصلی، نحوهٔ محاسبهٔ score، معیارهای ارزیابی، چالش‌ها، تصمیم‌های معماری و خروجی‌های نهایی را توضیح می‌دهد. تولید پاسخ با API و استناد به ناحیهٔ داخل تصویر، در دامنهٔ مرحلهٔ انجام‌شده قرار ندارند. خروجی فعلی، صفحه‌های مرتب شده همراه با شناسه و مسیر منبع است.

ساختار گزارش

بخش‌های آغازین به داده و مسیر فایل‌ها می‌پردازند. سپس استخراج متن، BM25،embedding تصویری، MaxSim و Hybrid تشریح می‌شوند. در ادامه برچسب‌ها، معیارها و نتایج می‌آیند و در پایان، راه‌حل چالش‌ها، گزینه‌های استفاده‌نشده و دستورهای بازتولید ارائه می‌شوند. تصاویر ترمینال و کد در کنار توضیح مربوط قرار گرفته‌اند.

۱ هدف فارسی سازی و تطبیق با بنچمارک اصلی

مقالهٔ FinRAGBench-V یک بنچمارک چند وجهی مالی با قابلیت استناد بصری معرفی می‌کند. corpus معرفی‌شده شامل ۶۰۷۸۰ صفحهٔ چینی و ۵۱۲۱۹ صفحهٔ انگلیسی است . نبود corpus و پرسش‌های فارسی در این ساختار، دلیل نیاز به آماده‌سازی مستقل دادهٔ فارسی بود. وجود یک مدل چندزبانه به‌تنهایی جای داده، metadata و مرجع ارزیابی فارسی را نمی‌گیرد.

برای فارسی‌سازی، منطق «صفحه به‌عنوان واحد بازیابی» حفظ شد. هر صفحهٔ PDF یک تصویر و یک شناسه دارد. پرسش فارسی به بردارهای مدل تبدیل می‌شود و نتیجهٔ بازیابی به همان صفحهٔ واقعی برمی‌گردد. این انتخاب به ما اجازه می‌دهد هم متن و هم عناصر بصری سند را در مسیر بازیابی تصویری نگه داریم، حتی اگر استخراج متن صفحه کامل نباشد.

جزء FinRAGBench V	معادل این پیاده سازی	وضعیت
PDF انگلیسی و چینی	گزارش‌های فارسی دیجی‌پی	دو PDF و ۱۲۲ صفحه
Corpus تصویری	PNG رنگی برای هر صفحه	ساخته و کنترل شد
Corpus Parquet	corpus-id و بایت image	۱۲۲ رکورد
پرسش‌های بنچمارک	questions_10.json	۱۰ سؤال محلی فارسی
ColQwen2	مدل ثابت با ورودی تصویر فارسی	بودجهٔ منتخب ۲۰۴۸
مرجع ارزیابی بازیابی	relevant_page_ids	برچسب صفحهٔ مرجع
ارزیابی بازیابی	Recall@10 و MRR@10	اسکریپت محلی مشترک
تولید پاسخ و استناد بصری	مرحلهٔ بعد	هنوز پیاده‌سازی نشده

قالب سؤال در اجرای نهایی JSON است؛ ساخت Query Parquet فارسی یا اجرای کامل ابزار TREC در این مرحله ادعا نمی شود. کاری که انجام دادیم، تطبیق بخش بازیابی و ساخت ارزیابی محلی با شناسه‌های معتبر corpus بود. بنابراین نتایج این مجموعه با نتایج مقالهٔ اصلی هم‌ارز یا مستقیماً قابل مقایسه نیستند.

۲ محیط اجرا و مبنای مسیرها

اجرا روی سرور homacloud با کاربر baseri و GPU مدل RTX 3090 انجام شد. اسکرین محیط، حافظهٔ کل ۲۴۵۷۶ MiB، درایور 570.124.06 و پشتیبانی CUDA نمایش‌دادهشدهٔ 12.8 را نشان می‌دهد. مقدار حافظهٔ مصرفی در این تصویر مربوط به حالت بیکار است؛ از آن نباید مصرف اوج هنگام encoding را نتیجه گرفت.

```
+-----------------------------------------------------------------------------------------+
| NVIDIA-SMI 570.124.06             Driver Version: 570.124.06     CUDA Version: 12.8     |
|-----------------------------------------+------------------------+----------------------+
| GPU  Name                 Persistence-M | Bus-Id          Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |           Memory-Usage | GPU-Util  Compute M. |
|                                         |                        |               MIG M. |
|=========================================+========================+======================|
|   0  NVIDIA GeForce RTX 3090        Off |   00000000:03:00.0 Off |                  N/A |
|  0%   28C    P0             80W /  350W |       1MiB /  24576MiB |      4%      Default |
|                                         |                        |                  N/A |
+-----------------------------------------+------------------------+----------------------+

+-----------------------------------------------------------------------------------------+
| Processes:                                                                              |
|  GPU   GI   CI              PID   Type   Process name                      GPU Memory |
|        ID   ID                                                             Usage      |
|=========================================================================================|
|  No running processes found                                                             |
+-----------------------------------------------------------------------------------------+
```

شکل ۱ مشخصات GPU و وضعیت بیکار دستگاه در زمان ثبت تصویر

نسخهٔ Python ثبت‌شده 3.10.12 است. در پایان، HEAD و origin/main روی کامیت 1a4f44d قرار داشتند. پوشهٔ داخلی پروژه مبنای دستورهاست و مخزن Git یک سطح بالاتر قرار دارد. این تفاوت در مسیرها علت بخشی از خطاهای اولیهٔ اجرا و فایل‌های ناخواستهٔ خارج از پوشهٔ داخلی بود.

```
Repository: /home/baseri/FinRAGBench-V-git
Project:    /home/baseri/FinRAGBench-V-git/FinRAGBench-V

cd /home/baseri/FinRAGBench-V-git/FinRAGBench-V
pwd
python --version
git status -sb
```

```
/home/baseri/FinRAGBench-V-git/FinRAGBench-V
a4f44d1 (HEAD -> main, origin/main, origin/HEAD) Compare Persian ColQwen2, BM25 and hybrid retrieval at k=10
## main...origin/main
Python 3.10.12
```

شکل ۲ پوشهٔ اجرا و نسخهٔ Python و تطابق main با origin/main

در دستورهای این گزارش، مسیرهای نسبی نسبت به Project بالا تفسیر می‌شوند. محیط Python اصلی در ترمینال فعال بود. دستور source .venv/bin/activate از پوشهٔ داخلی خطا داد، چون چنین پوشه‌ای آنجا وجود نداشت؛ ادامهٔ کار با محیط فعال و بررسی مسیر Python انجام شد، نه با ساخت مجدد یک محیط تصادفی.

۳ پوشه ها و فایل های اصلی پروژه

جدول زیر محل داده‌ها و فایل‌های اجرایی را مشخص می‌کند. برای ساخت مسیر مطلق هر رکورد، مسیر نسبی را به پوشهٔ Project در بخش قبل اضافه می‌کنیم. جدا نگه‌داشتن corpus، مدل، embedding و نتیجه کمک می‌کند خروجی بازیابی با ورودی encoding اشتباه نشود.

مسیر نسبی	نقش
data/fa/pdfs/fa-data/	PDFهای منبع دیجی‌پی
data/fa/images/	تصویر PNG هر صفحه
data/fa/pages.jsonl	شناسهٔ صفحه و منبع و ابعاد تصویر
retrieval/corpus/fa/corpus.parquet	corpus تصویری در قالب string و binary
data/fa/text/pages_text.jsonl	متن خام و نرمال‌شدهٔ صفحات
data/fa/evaluation/questions_10.json	پرسش‌ها و صفحات مرجع مشترک
retrieval/colqwen2-v1.0-merged/	فایل‌های مدل محلی ColQwen2
retrieval/encode/colqwen_fa_2048/	embedding منتخب و manifest و checkpoint
retrieval/encode/bm25_fa/index.pkl	index ذخیره‌شدهٔ BM25
models/reranker_manifest.json	شناسه و revision مدل Reranker
models/hf-cache/	cache محلی مدل مرتب‌سازی مجدد
retrieval/results/fa_comparison/	JSON و CSVهای مقایسهٔ نهایی

اسکریپت های مسئول هر مرحله

scripts_fa/prepare_digipay_fa_122pages.py
scripts_fa/extract_fa_text.py
scripts_fa/retrieve_fa_bm25.py
scripts_fa/encode_fa_corpus.py
scripts_fa/retrieve_fa.py
scripts_fa/retrieve_fa_hybrid.py
scripts_fa/evaluate_fa_retrieval.py
scripts_fa/evaluate_fa_compare.py

prepare دادهٔ تصویری را آماده می‌کند؛ extract متن را از PDF می‌خواند؛ encode بردارهای صفحه را می‌سازد. retrieveها برای جست‌وجو و بررسی تعاملی هستند. evaluate_fa_compare.py مبنای جدول نهایی سه روش است و از مدل‌ها و embedding موجود استفاده می‌کند؛ برای این مقایسه corpus دوباره encode نشد.

۴ انتخاب داده و تبدیل PDF به تصویر

دو گزارش دیجی‌پی انتخاب شدند، چون اسناد واقعی مالی فارسی با متن، عدد و نمودار هستند و مجموعه‌ای کوچک برای کنترل اولیهٔ مسیر توسعه فراهم می‌کنند. digipay-1400-fa.pdf دارای ۵۰ صفحه و digipay-1401-fa.pdf دارای ۷۲ صفحه است. در نتیجه، corpus نهایی ۱۲۲ صفحه دارد و هر صفحه یک واحد مستقل بازیابی است.

```
digipay-1400-fa.pdf | pages: 50
digipay-1401-fa.pdf | pages: 72
PNG images: 122
```

شکل ۳ شمارش واقعی صفحات دو PDF و تصاویر PNG خروجی

در prepare_digipay_fa_122pages.py، کتابخانهٔ PyMuPDF صفحه را با ۱۵۰ DPI، فضای رنگی RGB و بدون کانال alpha رندر می‌کند. بایت PNG هم روی دیسک ذخیره می‌شود و هم برای corpus در دسترس است. رندر صفحه، چیدمان نمودار و جدول و جهت بصری نوشتهٔ فارسی را حفظ می‌کند؛ کیفیت ورودی مدل بعداً به محدودیت resize و تعداد توکن تصویری هم وابسته است.

```python
4-
5-import pymupdf
6-import pyarrow as pa
7:import pyarrow.parquet as pq
8-from PIL import Image
9-
10-ROOT = Path(__file__).resolve().parents[1]
11-PDF_DIR = ROOT / "data/fa/pdfs/fa-data"
12-IMAGE_DIR = ROOT / "data/fa/images"
13:PARQUET_PATH = ROOT / "retrieval/corpus/fa/corpus.parquet"
14-METADATA_PATH = ROOT / "data/fa/pages.jsonl"
15:DPI = 150
16-
17-def main():
18-    pdfs = sorted(PDF_DIR.glob("*.pdf"))
19-    if len(pdfs) != 2:
20-        raise RuntimeError(f"Expected 2 PDFs, found {len(pdfs)}")
--
46-                page_id = f"fa_{pdf_path.stem}_p{page_number:04d}.png"
47-                image_path = IMAGE_DIR / page_id
48-
49:                pix = page.get_pixmap(
50:                    dpi=DPI,
51:                    colorspace=pymupdf.csRGB,
52-                    alpha=False,
53-                )
54-                image_bytes = pix.tobytes("png")
55-                image_path.write_bytes(image_bytes)
56-
--
66-                    "image_path": image_path.relative_to(ROOT).as_posix(),
67-                    "width": pix.width,
68-                    "height": pix.height,
69:                    "render_dpi": DPI,
```

شکل ۴ مسیرهای ورودی و خروجی و کد render با RGB و وضوح ۱۵۰ DPI

$$W_{px} \approx W_{pt}\frac{DPI}{72}, \qquad H_{px} \approx H_{pt}\frac{DPI}{72}$$

در فرمول، ابعاد PDF بر حسب point هستند و هر اینچ ۷۲ point دارد. این تبدیل، تخمین ابعاد تصویر قبل از گردکردن و محدودیت های renderer است. انتخاب ۱۵۰ DPI یک تنظیم اجرایی برای حفظ خوانایی و مهار حجم تصاویر بود؛ آزمایش کنترل‌شدهٔ مقایسهٔ چند DPI در این مرحله انجام نشده است.

۵ ساخت Parquet و کنترل سلامت corpus

فایل corpus.parquet ظرف ذخیرهٔ تصویر است، نه فایل embedding. schema آن دو ستون دارد: corpus-id از نوع string و image از نوع binary. در ستون image، بایت‌های PNG صفحه قرار گرفته‌اند. این ساختار، دادهٔ فارسی را برای مرحلهٔ encoding مشابه corpus تصویری آماده می‌کند.

```
Schema:
corpus-id: string
image: binary
Rows: 122
```

شکل ۵ schema فایل Parquet و تعداد ۱۲۲ رکورد

نوشتن فایل به‌صورت موقت و فشرده‌سازی Snappy انجام شد. پس از خواندن دوباره، تعداد رکوردها، یکتایی شناسه‌ها و schema کنترل شدند. علاوه بر این، تصویرها با PIL از نظر RGB بودن، ابعاد و تطابق بایت‌های داخل Parquet با فایل PNG روی دیسک بررسی شدند. تنها پس از این کنترل‌ها فایل موقت به خروجی نهایی تبدیل شد.

```
70-                })
71-
72-                if page_number % 10 == 0 or page_number == len(doc):
73-                    print(f"  Rendered {page_number}/{len(doc)}", flush=True)
74-
--
79-    table = pa.Table.from_pylist(rows, schema=schema)
80-
81-    # Write to a temporary file and validate before publishing it.
82:    temp_path = PARQUET_PATH.with_suffix(".parquet.tmp")
83-    pq.write_table(table, temp_path, compression="snappy")
84:    saved = pq.read_table(temp_path)
85-
86:    ids = saved["corpus-id"].to_pylist()
87:    assert saved.num_rows == total
88-    assert len(set(ids)) == total
89:    assert saved.schema.equals(schema)
90-
91-    for record, image_bytes in zip(
92:        metadata, saved["image"].to_pylist()
93-    ):
94-        with Image.open(BytesIO(image_bytes)) as image:
95:            assert image.mode == "RGB"
96-            assert image.size == (record["width"], record["height"])
97-            image.verify()
98-
99-        disk_path = ROOT / record["image_path"]
100-        assert disk_path.read_bytes() == image_bytes
--
106-    temp_path.rename(PARQUET_PATH)
107-
108-    print("\nValidation: OK")
109:    print("Pages:", saved.num_rows)
110-    print("Unique IDs:", len(set(ids)))
111:    print("Schema:", saved.schema)
112-    print("Parquet:", PARQUET_PATH)
113-    print("Metadata:", METADATA_PATH)
114-    print("Images:", IMAGE_DIR)
115-
116-if __name__ == "__main__":
```

شکل ۶ کد ذخیرهٔ Parquet و بررسی شناسه و schema و بایت تصویر

این کنترل از خطای خطرناک «بردار سالم برای صفحهٔ اشتباه» جلوگیری می‌کند. صرف وجود ۱۲۲ تصویر کافی نیست؛ ترتیب و شناسهٔ هر تصویر باید با corpus و metadata و بعداً embedding هماهنگ باشند. بررسی محتوا پیش از encoding نیز هزینهٔ تکرار یک اجرای GPU را کاهش می‌دهد.

۶ شناسهٔ صفحه و metadata فارسی

نام صفحه از نام PDF و شمارهٔ فیزیکی آن ساخته می‌شود. برای نمونه، fa_digipay-1401-fa_p0052.png به صفحهٔ فیزیکی ۵۲ گزارش ۱۴۰۱ اشاره دارد. پیشوند fa زبان را مشخص می‌کند و 0052p شمارهٔ صفحه را با چهار رقم نگه می‌دارد. مرجع سؤال باید دقیقاً با همین شناسه نوشته شود؛ شمارهٔ چاپ‌شدهٔ داخل سند ممکن است در بعضی PDFها با شمارهٔ فیزیکی متفاوت باشد.

```
page_id = f"fa_{pdf_path.stem}_p{page_number:04d}.png"
image_path = IMAGE_DIR / page_id
```

metadata در data/fa/pages.jsonl یک رکورد JSON برای هر صفحه نگه می‌دارد. source_pdf و pdf_page_number امکان بازگشت به منبع را فراهم می‌کنند. image_path مسیر تصویر رندرشده است و width و height و render_dpi برای کنترل آماده‌سازی استفاده می‌شوند.

```
{
  "corpus-id": "fa_digipay-1400-fa_p0001.png",
  "language": "fa",
  "source_pdf": "data/fa/pdfs/fa-data/digipay-1400-fa.pdf",
  "pdf_page_number": 1,
  "image_path": "data/fa/images/fa_digipay-1400-fa_p0001.png",
  "width": 1359,
  "height": 1359,
  "render_dpi": 150
}
```



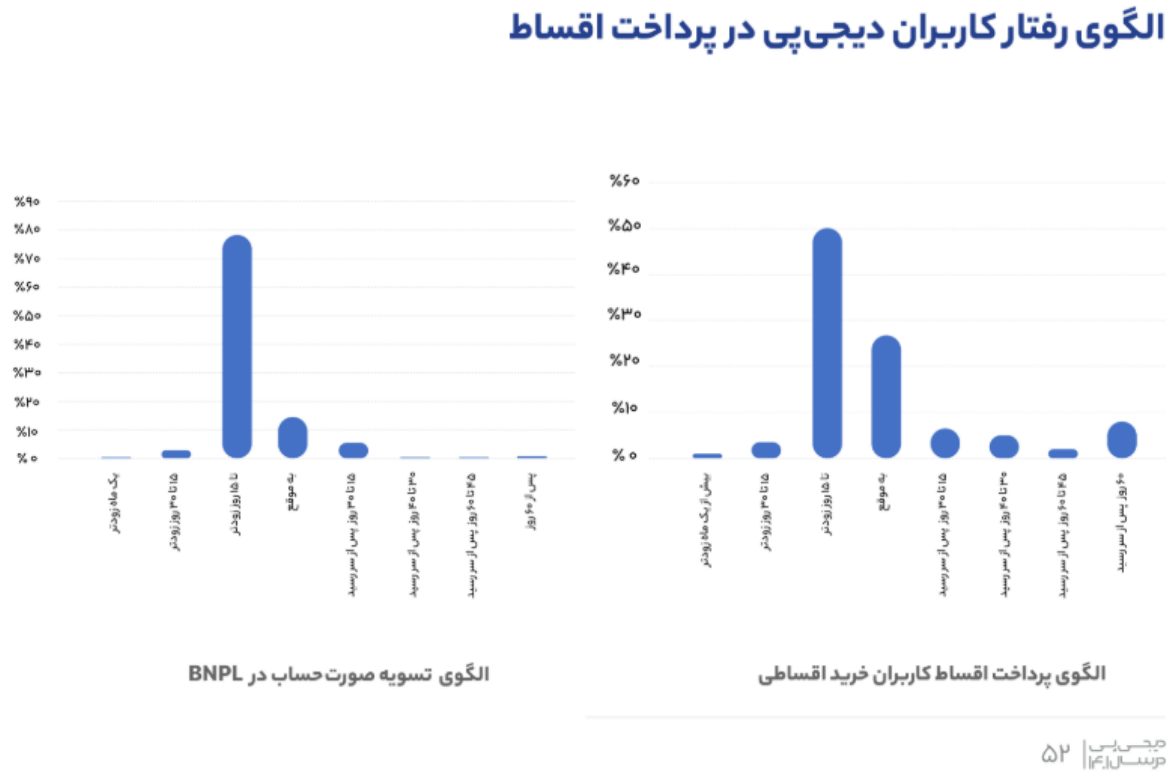
شکل ۷ نمونهٔ رکوردهای metadata در خروجی ترمینال

در مثال بالا متن رکورد برای خوانایی بازنویسی شده است؛ تصویر، شکل اصلی خروجی ترمینال را نشان می‌دهد. مسیر مطلق تصویر نمونه از اتصال ROOT با image_path ساخته می‌شود. در کدها استفاده از Path(__file__).resolve().parents[1] وابستگی به محل اجرای ترمینال را کاهش می‌دهد و ریشهٔ داخلی پروژه را ثابت نگه می‌دارد.

```
ROOT = Path(__file__).resolve().parents[1]
absolute_image_path = ROOT / record["image_path"]
```

۷ نمونهٔ واقعی صفحهٔ نمودار و چالش فارسی

سؤال 002_fa دربارهٔ نمودار رفتار کاربران دیجی‌پی در پرداخت اقساط است. مرجع ثبت‌شده، تصویر صفحهٔ ۵۲ گزارش ۱۴۰۱ است. در نمونهٔ صفحه، عنوان مربوط و دو نمودار اقساط و BNPL دیده می‌شوند؛ بنابراین مرجع از نظر محتوایی با موضوع سؤال سازگار است.



شکل ۸ صفحهٔ نمودار رفتار کاربران دیجی‌پی در پرداخت اقساط

برای اسناد فارسی باید میان نمایش صحیح صفحه و استخراج صحیح متن تفاوت گذاشت. ممکن است حروف و نمودار در تصویر کاملاً خوانا باشند، اما ترتیب متن استخراج‌شده مناسب خواندن انسان نباشد. راست‌به‌چپ بودن، ترکیب عدد و واژهٔ لاتین، ترتیب داخلی قطعه‌های متن و نگاشت فونت می‌توانند در این اختلاف نقش داشته باشند.

در این پروژه برای مسیر تصویری، رندر مستقیم صفحه این وابستگی به ترتیب استخراج متن را کاهش داد. برای مسیر 25BM، لایهٔ متنی جدا خوانده و نرمال شد. هیچ نتیجه‌ای دربارهٔ خراب‌بودن فونت همهٔ صفحات یا اسکن‌بودن همهٔ PDFها گرفته نشد؛ هر مشکل باید در همان صفحه بررسی شود.

صفحهٔ مرجع در 2ColQwen رتبهٔ ۱۹ داشت، ولی این به‌تنهایی خرابی corpus یا embedding را اثبات نمی‌کند. این یک خطای بازیابی در آن سؤال است. نمونهٔ تصویری به ما کمک کرد صحت مرجع را از کیفیت رتبه‌بندی مدل جدا بررسی کنیم.

۸ استخراج متن برای 25BM از لایهٔ PDF

متن 25BM از تصویر PNG استخراج نشده است. scripts_fa/extract_fa_text.py با PyMuPDF خود PDF منبع را باز می‌کند و صفحه را بر اساس metadata می‌خواند. سپس page.get_text("text", sort=False) متن موجود در PDF را استخراج می‌کند. OCR در این مرحله اجرا نشده است.

documents یک cache کوچک از PDFهای بازشده است تا برای هر صفحه فایل دوباره باز نشود. شمارهٔ صفحه در metadata از یک شروع می‌شود، اما اندیس PyMuPDF از صفر؛ به همین دلیل هنگام دسترسی به صفحه، یک واحد کم می‌شود. این تبدیل برای جلوگیری از جابه‌جایی متن و تصویر ضروری است.

```python
from pathlib import Path
import json
import unicodedata
import pymupdf

ROOT = Path(__file__).resolve().parents[1]
METADATA = ROOT / "data/fa/pages.jsonl"
OUTPUT = ROOT / "data/fa/text/pages_text.jsonl"

def normalize(text):
    text = unicodedata.normalize("NFKC", text)
    text = text.translate(str.maketrans({"ي": "ی", "ك": "ک"}))
    return text

def main():
    records = [
        json.loads(line)
        for line in METADATA.read_text(encoding="utf-8").splitlines()
        if line.strip()
    ]
    OUTPUT.parent.mkdir(parents=True, exist_ok=True)

    documents = {}
    extracted = []

    try:
        for record in records:
            pdf_path = ROOT / record["source_pdf"]
            if pdf_path not in documents:
                documents[pdf_path] = pymupdf.open(pdf_path)

            page = documents[pdf_path][record["pdf_page_number"] - 1]
            raw_text = page.get_text("text", sort=False)
            text = normalize(raw_text)

            extracted.append({
                **record,
                "text_raw": raw_text,
                "text_normalized": text,
                "text_chars": len(text.strip()),
```

شکل ۹ کد خواندن PDF و استخراج متن و ساخت رکورد متن صفحه

```
page = documents[pdf_path][record["pdf_page_number"] - 1]
raw_text = page.get_text("text", sort=False)
text = normalize(raw_text)
```

انتخاب sort=False متن را بدون تحمیل مرتب‌سازی هندسی اضافی می‌خواند. این تنظیم اصلاح کامل RTL نیست و خروجی باید با نمونهٔ واقعی بررسی شود. مرحلهٔ normalize از NFKC و تبدیل حروف عربی ي و ك به معادل فارسی ی و ک استفاده می‌کند. متن خام هم نگه داشته می‌شود تا اثر پاک‌سازی قابل بررسی بماند.

۹ ذخیرهٔ متن و پوشش صفحات قابل جست وجو

خروجی استخراج در data/fa/text/pages_text.jsonl ذخیره می‌شود. هر رکورد علاوه بر metadata شامل text_raw،text_normalized و text_chars است. کد در پایان PDFهای بازشده را می‌بندد، تعداد صفحات با متن خالی را گزارش می‌کند و نمونهٔ متن چند صفحه را برای بررسی ذخیره یا چاپ می‌کند.

```
            })
    finally:
        for document in documents.values():
            document.close()

    with OUTPUT.open("w", encoding="utf-8") as f:
        for record in extracted:
            f.write(json.dumps(record, ensure_ascii=False) + "\n")

    print("Saved pages:", len(extracted))
    print("Pages with no extracted text:",
          sum(r["text_chars"] == 0 for r in extracted))
    print("Output:", OUTPUT)

    samples = [
        "fa_digipay-1400-fa_p0003.png",
        "fa_digipay-1401-fa_p0052.png",
    ]
    for page_id in samples:
        record = next(r for r in extracted if r["corpus-id"] == page_id)
        sample_path = OUTPUT.parent / f"{Path(page_id).stem}.txt"
        sample_path.write_text(
            record["text_normalized"], encoding="utf-8"
        )
        print("\nSample:", page_id)
        print(record["text_normalized"][:2000])
        print("Sample file:", sample_path)

if __name__ == "__main__":
    main()
```

شکل ۱۰ بستن PDFها و ذخیرهٔ JSONL و نمونهٔ متن استخراج‌شده

```
Searchable pages: 112
Tokenizer version: fa_v1
Total text records: 122

Page: fa_digipay-1401-fa_p0052.png
Normalized text preview:

ا ق ا م ا ط ا ن ا ر ی ز ا ر ب ا د ر چ ا ت خ ا د ر چ ا ر د ی ی ی ی د ن ا ر ی ز ا ر ا ک ر ا ف ر ی و گ ا
ا ق ا م ا ط ا ن ا ر ی ز ا ر ا ک ط ا ق ا م ا ط ا ن ا ر ی خ د ی ن ا ر ی ز ا ر ا ک ط ا ق ا م ا ط ا BNPL ا ی و گ ا ر د ب ا م ح ر و م ه ی و م ی ت ی و گ ا
ر چ ی ب
ر چ د و ز ه ا م ک ی ز ا
۱۵ ا ت ۳۰ ر چ د و ز ز و ر
ا ت ۱۵ ر چ د و ز ز و ر
ع ق ی و م ه ب
۱۵ ا ت ۳۰ د ی ی ی ر ر ی ز ا ر ی چ ز و ر
۴۵ ا ت ۶۰ د ی ی ی ر ر ی ز ا ر ی چ ز و ر
۶۰ د ی ی ی ر ر ی ز ا ر ی چ ز و ر
۳۰ ا ت ۴۰ د ی ی ی ر ر ی ز ا ر ی چ ز و ر
٪ ۰
٪ ۱۰
٪ ۲۰
٪ ۳۰
٪ ۴۰
٪ ۵۰
٪ ۶۰
ر چ د و ز ه ا م ک ی
۱۵ ا ت ۳۰ ر چ د و ز ز و ر
ا ت ۱۵ ر چ د و ز ز و ر
ع ق ی و م ه ب
۱۵ ا ت ۳۰ د ی ی ی ر ر ی ز ا ر ی چ ز و ر
۴۵ ا ت ۶۰ د ی ی ی ر ر ی ز ا ر ی چ ز و ر
ز و ر ی چ ۶۰ ز ا
۳۰ ا ت ۴۰ د ی ی ی ر ر ی ز ا ر ی چ ز و ر
٪ ۰
٪ ۱۰
٪ ۲۰
٪ ۳۰
```

شکل ۱۱ ۱۲۲ رکورد متن و ۱۱۲ صفحهٔ قابل جست‌وجو در index فارسی

در corpus تصویری ۱۲۲ صفحه باقی مانده است، اما 25index BM فقط ۱۱۲ صفحهٔ دارای توکن قابل جست‌وجو دارد. ۱۰ صفحهٔ دیگر از corpus اصلی حذف نشده‌اند؛ در مسیر متنی قابل بازیابی نیستند. نبود توکن، لزوماً به معنای اسکن‌بودن صفحه نیست و می‌تواند ناشی از صفحهٔ خالی، محتوای تصویری یا محدودیت استخراج باشد.

۱۰ نرمال سازی و توکن سازی فارسی در 25BM

توکن‌ساز 1fa_v در retrieve_fa_bm25.py برای سؤال و متن صفحه یکسان استفاده می‌شود. NFKC و lowercase اجرا می‌شوند؛ ي و ك به ی و ک تبدیل می‌شوند؛ رقم‌های فارسی و عربی به رقم‌های لاتین می‌روند. نیم‌فاصله به فاصله تبدیل می‌شود و zero-width joiner، اعراب ترکیبی و کشیده حذف می‌شوند. سپس عبارت منظم، دنباله‌های حرف و عدد را به توکن تبدیل می‌کند.

```python
from pathlib import Path
from datetime import datetime, timezone
from collections import Counter
import argparse
import hashlib
import json
import pickle
import re
import unicodedata

from rank_bm25 import BM25Okapi

ROOT = Path(__file__).resolve().parents[1]
SOURCE = ROOT / "data/fa/text/pages_text.jsonl"
INDEX = ROOT / "retrieval/encode/bm25_fa/index.pkl"
RESULTS = ROOT / "retrieval/results/fa_bm25"
TOKENIZER_VERSION = "fa_v1"

def tokenize(text):
    text = unicodedata.normalize("NFKC", text).lower()
    text = text.translate(str.maketrans({
        "ي": "ی", "ك": "ک",
        **dict(zip("۰۱۲۳۴۵۶۷۸۹", "0123456789")),
        **dict(zip("٠١٢٣٤٥٦٧٨٩", "0123456789")),
    }))
    text = text.replace("\u200c", " ").replace("\u200d", "")
    text = "".join(
        c for c in text
        if unicodedata.category(c) != "Mn" and c != "\u0640"
    )
    return re.findall(r"[^\W_]+", text, flags=re.UNICODE)

def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--topk", type=int, default=10)
    args = parser.parse_args()
    if args.topk < 1:
        raise ValueError("topk must be positive")

    content = SOURCE.read_bytes()
```

شکل ۱۲ پیاده‌سازی tokenize و تعریف مسیر index و نسخهٔ 1fa_v

علت این کار کاهش اختلاف‌های نوشتاری است: «دیجی‌پی» و «دیجی پی» یا «۱۴۰۱» و «1401» نباید صرفاً به دلیل Unicode و شکل رقم، دو نمایش ناسازگار داشته باشند. این پاک‌سازی برای تطبیق واژگانی 25BM مهم است. توکن‌ساز فعلی ریشه‌یابی، تشخیص مترادف یا بازسازی کامل ترتیب متن PDF انجام نمی‌دهد.

```
text = text.replace("\u200c", " ").replace("\u200d", "")
return re.findall(r"[^\W_]+", text, flags=re.UNICODE)
```

نام نسخهٔ tokenizer در index ثبت می‌شود. اگر قواعد تغییر کنند، index قبلی دیگر با پرسش‌های جدید سازگار فرض نمی‌شود. برای مقایسهٔ قابل تکرار، تغییر توکن‌ساز باید همراه با بازسازی index و ارزیابی دوباره باشد.

۱۱ ساخت و استفادهٔ دوباره از index متنی

index در retrieval/encode/bm25_fa/index.pkl نگه داشته می‌شود. پیش از استفاده، 256SHA فایل متن و نسخهٔ tokenizer با مقادیر ذخیره‌شده مقایسه می‌شوند. اگر تطابق وجود داشته باشد، index بارگذاری می‌شود؛ در غیر این صورت اسکریپت 25BM آن را از رکوردهای دارای توکن می‌سازد. این cache با cache دانلود مدل متفاوت است.

```
    fingerprint = hashlib.sha256(content).hexdigest()
    state = None

    if INDEX.exists():
        with INDEX.open("rb") as f:
            cached = pickle.load(f)
        if (
            cached["source_sha256"] == fingerprint
            and cached["tokenizer_version"] == TOKENIZER_VERSION
        ):
            state = cached
            print("Loaded saved BM25 index.")

    if state is None:
        records = [
            json.loads(line)
            for line in content.decode("utf-8").splitlines()
            if line.strip()
        ]
        ids = [r["corpus-id"] for r in records]
        if len(ids) != len(set(ids)):
            raise RuntimeError("Duplicate page IDs")

        # Pages without searchable text remain in the source dataset,
        # but cannot participate in this text-only retrieval.
        records = [
            r for r in records if tokenize(r["text_normalized"])
        ]
        if not records:
            raise RuntimeError("No searchable text found")
```

شکل ۱۳ بررسی fingerprint و نسخهٔ tokenizer و حذف صفحات بدون توکن از index

```
            raise RuntimeError("No searchable text found")

        tokens = [tokenize(r["text_normalized"]) for r in records]
        state = {
            "source_sha256": fingerprint,
            "tokenizer_version": TOKENIZER_VERSION,
            "records": records,
            "model": BM25Okapi(tokens),
            "vocabulary": set(word for page in tokens for word in page),
        }
        INDEX.parent.mkdir(parents=True, exist_ok=True)
        temporary = INDEX.with_suffix(".tmp")
        with temporary.open("wb") as f:
            pickle.dump(state, f, protocol=4)
        temporary.replace(INDEX)
        print("Built and saved BM25 index:", INDEX)

records = state["records"]
print("Searchable pages:", len(records))
print("Pages per PDF:", dict(Counter(r["source_pdf"] for r in records)))
RESULTS.mkdir(parents=True, exist_ok=True)

while True:
    try:
        query = input("\nQuestion (exit to finish): ").strip()
    except (EOFError, KeyboardInterrupt):
```

شکل ۱۴ ساخت BM25Okapi و ذخیرهٔ اتمیک index با فایل موقت

مدل BM25Okapi، رکوردهای قابل جست‌وجو و vocabulary داخل pickle ذخیره می‌شوند. نوشتن ابتدا در فایل موقت و سپس replace انجام می‌شود تا فایل نیمه‌نوشته به‌عنوان index نهایی استفاده نشود. در ارزیابی مشترک، index ناسازگار به خطا منجر می شود تا نتیجه با دادهٔ قدیمی محاسبه نشود.

۱۲ نحوهٔ محاسبهٔ score در 25BM

25BM بر میزان تطبیق واژگان سؤال و صفحه تکیه می‌کند. واژهٔ کم‌تکرار در corpus معمولاً وزن بیشتری می‌گیرد، تکرار واژه در صفحه با افزایش فراوانی به اشباع می‌رسد و طول صفحه در امتیاز اثر دارد. فرمول زیر با ساختار BM25Okapi مورد استفاده سازگار است .

$$S_{BM25}(q,d) = \sum_{t \in q} IDF(t) \frac{f(t,d)(k_1+1)}{f(t,d) + k_1(1 - b + b|d|/\bar{L})}$$

نماد	معنا
q و d	سؤال و متن صفحه
f(t,d)	تعداد رخداد توکن t در صفحه
||d|| و L̄	طول توکنی صفحه و میانگین طول صفحات index
N و n_t	تعداد صفحات index و تعداد صفحات دارای توکن t
1k و b	اشباع فراوانی و اثر نرمال‌سازی طول

$$IDF_0(t) = \ln \frac{N - n_t + 0.5}{n_t + 0.5}$$

کد BM25Okapi بدون پارامتر سفارشی فراخوانی شده است. پیش‌فرض‌های کتابخانه 1.5=1k و 0.75=b و 0.25=epsilon هستند. در این پیاده‌سازی، IDF اولیهٔ منفی با epsilon ضربدر میانگین IDF اولیه جایگزین می‌شود؛ بنابراین فرمول رایج دارای log(1+...) را نباید به‌عنوان فرمول دقیق همین کتابخانه نوشت .

```
tokens = [tokenize(r["text_normalized"]) for r in records]
model = BM25Okapi(tokens)
scores = model.get_scores(tokenize(query))
```

رتبه‌بندی از score بزرگ‌تر به کوچک‌تر انجام می‌شود. در اسکریپت مقایسه، برابری score با شناسهٔ صفحه به‌صورت صعودی شکسته می‌شود. اگر هیچ توکن سؤال در vocabulary نباشد، نتیجهٔ 25BM خالی است. برای نامزدهای متنی Hybrid، تنها صفحات دارای اشتراک توکنی با سؤال پذیرفته می‌شوند. score خام 25BM درصد یا احتمال نیست و با score خام MaxSim قابل جمع مستقیم نیست.

۱۳ embedding تصویری 2ColQwen با بودجهٔ ۲۰۴۸

در مسیر تصویری، صفحه به مجموعه‌ای از بردارها تبدیل می‌شود. برخلاف یک embedding واحد برای کل صفحه، خروجی 2ColQwen چندبرداری است و هر بردار ۱۲۸ بعد دارد. فایل منتخب در 122-0.0.retrieval/encode/colqwen_fa_2048/embeddings.corpus.rank قرار دارد و manifest.json مشخصات ساخت آن را نگه می‌دارد.

```
image_token_budget : 2048
max_pixels : 1605632
batch_size : 1
dtype : float16
attention : sdpa
pages : 122
embedding_file : embeddings.corpus.rank.0.0-122
Shape counts: {(2036, 128): 49, (2011, 128): 1, (2025, 128): 72}
Embedding size MiB: 60.48
```

شکل ۱۵ تنظیمات manifest و شکل بردارها و حجم فایل embedding منتخب

ویژگی	مقدار
بودجهٔ توکن تصویری	۲۰۴۸
حداکثر پیکسل	۱۶۰۵۶۳۲
batch ساخت corpus	۱ صفحه
dtype و attention	16float و sdpa
تعداد صفحات	۱۲۲
شکل بردارهای ۴۹ صفحه	(2036, 128)
شکل بردارهای ۱ صفحه	(2011, 128)
شکل بردارهای ۷۲ صفحه	(2025, 128)
حجم فایل ثبت‌شده	۶۰٫۴۸ MiB

$$P_{max} = 2048 \times 28 \times 28 = 1605632$$

۲۰۴۸ سقف تنظیم ورودی تصویر است، نه الزام به دقیقاً ۲۰۴۸ بردار برای هر صفحه. ابعاد صفحه و resize و grid پردازشگر بر طول واقعی خروجی اثر دارند. بنابراین شکل‌های ۲۰۳۶ و ۲۰۲۵ با بودجهٔ ثبت‌شده تناقض ندارند. عدد ۱۲۸ بعد هر بردار است و نباید با تعداد توکن اشتباه شود.

$$Memory_{raw} \approx \sum_{p=1}^{122} n_p \times 128 \times 2 \quad bytes$$

فرمول حافظه تخمین حجم خام بردارهای 16float است؛ سربار pickle، وزن مدل، tensorهای میانی و حافظهٔ موقت GPU را شامل نمی‌شود. حجم embedding روی دیسک شاخص مصرف اوج حافظهٔ encoding نیست.

۱۴ تنظیمات encoding و جلوگیری از بازنویسی

encode_fa_corpus.py با آرگومان --image-tokens مقدار پیش‌فرض ۲۰۴۸ را می‌گیرد و مسیر خروجی را بر اساس همان مقدار انتخاب می‌کند. max_pixels از TOKENS × 28 × 28 محاسبه می‌شود. نام پوشهٔ 2048_colqwen_fa به تنهایی تضمین تنظیم صحیح نیست؛ manifest و تنظیمات ذخیره‌شده، معیار بررسی هستند.

```
27-
28-def main():
29-    parser = argparse.ArgumentParser()
30:    parser.add_argument("--image-tokens", type=int, default=2048)
31-    parser.add_argument("--output-dir", default=None)
32-    args = parser.parse_args()
33-    if args.image_tokens < 1:
34:        raise ValueError("image-tokens must be positive")
35-
36-    TOKENS = args.image_tokens
37-    OUT = (
38-        ROOT / args.output_dir if args.output_dir
39-        else ROOT / f"retrieval/encode/colqwen_fa_{TOKENS}"
40-    )
41:    PARTS = OUT / "page_checkpoints"
42-    FINAL = OUT / "embeddings.corpus.rank.0.0-122"
43-
44-    if not torch.cuda.is_available():
45-        raise RuntimeError("CUDA is not available")
46:    if FINAL.exists():
47-        raise RuntimeError("Final embeddings already exist; refusing to overwrite")
48-
49-    records = pq.read_table(CORPUS).to_pylist()
50-    ids = [r["corpus-id"] for r in records]
--
56-        "image_token_budget": TOKENS,
57-        "max_pixels": TOKENS * 28 * 28,
58-        "batch_size": 1,
59:        "dtype": "float16",
60:        "attention": "sdpa",
61-        "model_files": {
62-            p.name: {
63-                "size": p.stat().st_size,
64-                "mtime_ns": p.stat().st_mtime_ns,
--
67-            if p.is_file() and p.suffix in {".json", ".safetensors", ".bin"}
68-        },
69-    }
```

شکل ۱۶ آرگومان بودجهٔ تصویر و مسیر checkpoint و تنظیمات 16FP و SDPA

وجود CUDA پیش از اجرا کنترل می‌شود. اگر فایل embedding نهایی از قبل وجود داشته باشد، اسکریپت از بازنویسی آن خودداری می‌کند. این رفتار برای حفظ نسخهٔ منتخب و جلوگیری از جایگزینی ناخواستهٔ خروجی با تنظیمات دیگر مفید بود. batch_size=1 نیز فشار حافظهٔ ساخت embedding را کاهش می‌دهد.

مشخصات فایل‌های مدل، از جمله size و mtime_ns، در تنظیمات ثبت شده‌اند. این اطلاعات و corpus_sha256 برای تشخیص تغییر ورودی کاربرد دارند؛ size و mtime به‌تنهایی هش محتوای مدل یا اثبات کامل یکسان‌بودن وزن‌ها نیستند. برای بازتولید قوی‌تر، revision ثابت و هش محتوای فایل وزن هم مفید است.

۱۵ checkpoint صفحه ای و ادامهٔ encoding

برای جلوگیری از تکرار encoding پس از قطع اجرا، هر صفحه checkpoint مستقل دارد. پوشهٔ page_checkpoints/ در همان مسیر embedding است و resume_settings.json تنظیمات اجرای قابل ادامه را نگه می‌دارد. تنظیمات جدید باید با تنظیمات ذخیره‌شده سازگار باشند؛ ترکیب checkpointهای متعلق به بودجه یا مدل متفاوت می‌تواند خروجی ناسازگار بسازد.

```
70:    settings_path = OUT / "resume_settings.json"
71:    if settings_path.exists():
72-        previous = json.loads(settings_path.read_text())
73-        if previous != settings:
74-            raise RuntimeError("Resume settings changed; use a new output folder")
75-    else:
--
78-    print("Loading model with SDPA...", flush=True)
79-    model = ColQwen2.from_pretrained(
80-        str(MODEL),
81:        torch_dtype=torch.float16,
82-        device_map="cuda:0",
83:        attn_implementation="sdpa",
84-    ).eval()
85-    processor = ColQwen2Processor.from_pretrained(str(MODEL))
86-
87-    processor.max_pixels = settings["max_pixels"]
--
97-          type(model.visual.blocks[0].attn).__name__, flush=True)
98-
99-    for index, record in enumerate(records):
100:        checkpoint = PARTS / f"page_{index:04d}.pkl"
101:        if checkpoint.exists():
102:            with checkpoint.open("rb") as f:
103-                saved = pickle.load(f)
104-            if saved["page_id"] != record["corpus-id"]:
105-                raise RuntimeError("Checkpoint ID mismatch")
106-            vector = saved["embedding"]
107-            if vector.ndim != 2 or not torch.isfinite(vector).all().item():
108:                raise RuntimeError("Invalid checkpoint")
109-            print(f"Reused {index + 1}/122", flush=True)
110-            del saved, vector
111-            continue
112-
--
123-            if vector.ndim != 2 or not torch.isfinite(vector).all().item():
124-                raise RuntimeError("Invalid embedding")
125-
```

شکل ۱۷ کنترل تنظیمات ادامهٔ اجرا و بارگذاری مدل رسمی و اعتبارسنجی checkpoint

```
126:            save_atomic(checkpoint, {
127-                "page_id": record["corpus-id"],
128-                "embedding": vector,
129-                "image_grid_thw": grid,
130-            })
```

شکل ۱۸ ذخیرهٔ page_id و embedding و image_grid در checkpoint

در checkpoint موجود، page_id، دوبعدی‌بودن embedding و finite بودن مقادیر بررسی می‌شود. صفحهٔ سالم دوباره محاسبه نمی‌شود و صفحهٔ جدید با ذخیرهٔ اتمیک ثبت می‌شود. این راه‌حل، اجرای طولانی را قابل ادامه کرد و احتمال مصرف دوبارهٔ زمان GPU برای صفحات تکمیل‌شده را کاهش داد.

کد encoder نشان می‌دهد از ColQwen2 و ColQwen2Processor در colpali_engine استفاده شده است. اسکریپت ارزیابی نهایی برای پرسش و score از wrapper ColQwen2Model در OpenMatch استفاده می‌کند. این دو بخش باید دقیقاً با همین نقش معرفی شوند؛ سازگاری در اجرای فعلی بررسی شده، اما تعویض آزاد هر نسخهٔ کتابخانه یا processor بدون کنترل تنظیمات مجاز فرض نمی‌شود.

۱۶ بازیابی تصویری و محاسبهٔ MaxSim

برای هر سؤال، processor.process_queries ورودی متنی مدل را می‌سازد و مدل بردارهای توکنی سؤال را تولید می‌کند. سپس score میان این بردارها و بردارهای صفحات محاسبه می‌شود. در MaxSim، هر بردار سؤال بهترین شباهت خود را میان بردارهای صفحه پیدا می‌کند و این بهترین شباهت‌ها جمع می‌شوند .

$$S_{MaxSim}(q,p) = \sum_{i=1}^{m} \max_{1 \le j \le n_p} \langle Q_i, D_{p,j} \rangle$$

Q_i بردار توکن i سؤال، D_p,j بردار j صفحهٔ m،p تعداد بردارهای سؤال و n_p تعداد بردارهای آن صفحه است. ضرب داخلی، میزان شباهت بردارها را می‌دهد؛ برای بردارهای نرمال‌شده با 2L، ضرب داخلی همان شباهت کسینوسی است. جزئیات mask و padding به processor سپرده شده‌اند و این فرمول منطق امتیاز را بیان می‌کند.

قطعه کد ارزیابی تصویری

```
# scripts_fa/evaluate_fa_compare.py
inputs = model.processor.process_queries(
   [question['query']]
).to('cuda:0')
query_vectors = model.model(**inputs)
scores = model.processor.score(
   query_vectors, vectors,
   device='cuda', batch_size=batch_size
)[0].detach().float().cpu()
order = sorted(
   range(len(ids)),
   key=lambda i: (-float(scores[i]), ids[i])
)
```

score خام MaxSim احتمال نیست و می‌تواند از یک بزرگ‌تر باشد؛ جمع چند شباهت توکنی چنین دامنه‌ای ایجاد می‌کند. در سؤال fa_002، بهترین score برابر ۲۷٫۴۵۳۱ بود و صفحهٔ مرجع رتبهٔ ۱۹ گرفت. از بزرگی این عدد نمی‌توان درصد موفقیت یا ارتباط معنایی قطعی برداشت کرد. مقایسهٔ رتبه‌ها برای همان سؤال مفیدتر از مقایسهٔ خام score سؤال‌های متفاوت است.

اسکریپت ۱۲۲ شناسهٔ یکتا، تطابق با metadata،shape ثبت‌شده در manifest، بعد ۱۲۸ و finite بودن scoreها را کنترل می‌کند. همهٔ صفحات امتیاز می‌گیرند و سپس ده نتیجهٔ اول برای معیار نهایی انتخاب می‌شوند. score-batch-size پیش‌فرض ۱۶ مربوط به امتیازدهی است و با batch_size=1 مرحلهٔ encoding تفاوت دارد.

۱۷ مسیر Hybrid و ادغام نامزدها

Hybrid برای استفاده از مکمل‌بودن دو مسیر ساخته شد. 2ColQwen می‌تواند از ساختار بصری صفحه استفاده کند و 25BM تطبیق واژگان را فراهم می‌کند. در مقایسهٔ نهایی، ۲۰ نامزد اول مسیر تصویری و حداکثر ۲۰ نامزد متنی با اشتراک توکنی انتخاب می‌شوند. اجتماع شناسه‌ها، نامزدهای تکراری را حذف می‌کند؛ در نتیجه تعداد نامزدهای یکتا حداکثر ۴۰ است.

$$C(q) = Top_{20}^{visual}(q) \cup Top_{20}^{BM25}(q), \qquad |C(q)| \leq 40$$

نامزدها با score خام دو روش جمع یا میانگین‌گیری نمی‌شوند. یک Reranker متنی زوج سؤال و متن هر صفحه را می‌خواند و logit ارتباط تولید می‌کند. ترتیب نهایی براساس همین logit تعیین می‌شود. مسیر بارگذاری مدل از models/reranker_manifest.json می‌آید و cache محلی در /models/hf-cache است. مدل مورد استفاده در مسیر توسعه 3BAAI/bge-reranker-v2-m بود؛ manifest، مرجع اجرایی شناسه و revision است .

```
152:def visual_rankings(questions, embedding_dir, batch_size, pages):
153-    import torch
154-    sys.path.insert(0, str(ROOT / 'src'))
155-    from openmatch.modeling.modeling_colqwen.modeling_colqwen_model import ColQwen2Model
156-    if not torch.cuda.is_available():
157-        raise RuntimeError('CUDA is not available')
158-    manifest_path = embedding_dir / 'manifest.json'
159-    manifest = json.loads(manifest_path.read_text(encoding='utf-8'))
160-    if manifest.get('image_token_budget') != 2048:
161-        raise ValueError('Expected existing 2048-token corpus embeddings')
162-    files = sorted(p for p in embedding_dir.glob('embeddings.corpus.rank.*')
163-                   if p.is_file() and not p.name.endswith('.tmp'))
164-    if not files:
--
214:def hybrid_rankings(questions, visual, text_candidates, pages, candidate_count, batch_size):
215-    import torch
216-    from transformers import AutoTokenizer, AutoModelForSequenceClassification
217-    manifest_path = ROOT / 'models/reranker_manifest.json'
218-    manifest = json.loads(manifest_path.read_text(encoding='utf-8'))
219-    options = {'cache_dir': str(ROOT / 'models/hf-cache'), 'local_files_only': True}
220-    if manifest.get('revision'):
221-        options['revision'] = manifest['revision']
222-    print('Loading local text reranker...', flush=True)
223-    tokenizer = AutoTokenizer.from_pretrained(manifest['model_id'], **options)
224-    reranker = AutoModelForSequenceClassification.from_pretrained(
225-        manifest['model_id'], torch_dtype=torch.float16, **options).to('cuda:0').eval()
226-    output = {}
```

شکل ۱۹ بررسی بودجهٔ ۲۰۴۸ در مسیر تصویری و بارگذاری محلی Reranker

در evaluate_fa_compare.py، نتیجهٔ تصویری یک بار محاسبه و برای شاخهٔ 2ColQwen و ساخت نامزدهای Hybrid مشترک استفاده می‌شود. این تصمیم اختلاف ناشی از استفادهٔ دو embedding جدا در مقایسه را حذف کرد. نسخهٔ تعاملی قدیمی retrieve_fa_hybrid.py مسیر آزمایشی colpali_fa_test را می‌خواند؛ جدول نهایی به آن اجرا وابسته نیست و مبنایش اسکریپت مقایسهٔ مشترک است.

۱۸ امتیازدهی Reranker و محدودیت متن صفحه

$$z(q, p) = f_\theta(q, text(p))$$

در این فرمول، fθ مدل مرتب‌سازی مجدد و z امتیاز خام زوج سؤال و متن صفحه است.
AutoModelForSequenceClassification خروجی logits را می‌دهد و کد آن را به float تبدیل می‌کند. استفاده از زوج سؤال و صفحه، امتیاز ارتباط مستقیم تولید می‌کند؛ این مرحله embedding ثابت صفحه برای جست‌وجوی مستقل نمی‌سازد.

```
231:        candidate_ids = sorted(set(visual_ids) | set(text_ids))
232-        usable = [p for p in candidate_ids if pages[p]['text_normalized'].strip()]
233-        textless = sorted(set(candidate_ids) - set(usable))
234-        scores, truncated = {}, []
235-        for pid in usable:
236-            length = len(tokenizer(query, pages[pid]['text_normalized'],
237-                                   truncation=False)['input_ids'])
238-            if length > 512:
239-                truncated.append(pid)
240-        for offset in range(0, len(usable), batch_size):
241-            batch_ids = usable[offset:offset + batch_size]
242-            inputs = tokenizer([[query, pages[p]['text_normalized']] for p in batch_ids],
243-                               padding=True, truncation='only_second', max_length=512,
--
251:        ranked = sorted(usable, key=lambda p: (-scores[p], p)) + textless
252-        vranks = {p: i for i, p in enumerate(visual_ids, 1)}
253-        tranks = {p: i for i, p in enumerate(text_ids, 1)}
254-        details = [{'rank': i, 'page_id': p, 'reranker_score': scores.get(p),
255-                    'visual_rank': vranks.get(p), 'bm25_rank': tranks.get(p)}
256-                   for i, p in enumerate(ranked, 1)]
257-        row = evaluated_row(question, ranked, details)
258-        row.update({'candidate_count': len(candidate_ids), 'candidate_page_ids': candidate_ids,
259-                    'textless_pages': textless, 'truncated_pages': truncated})
260-        output[qid] = row
261-        print('Hybrid scored:', qid, flush=True)
262-    del reranker, tokenizer
263-    gc.collect()
```

شکل ۲۰ ادغام نامزدها و محدودیت ۵۱۲ و ترتیب نهایی براساس score Reranker

زوج ورودی به حداکثر ۵۱۲ توکن محدود می‌شود و truncation="only_second" بر متن صفحه اعمال می‌شود. این انتخاب برای مهار حافظه و هزینهٔ reranking است و با بودجهٔ تصویری ۲۰۴۸ تفاوت دارد. صفحه‌های دارای متن طولانی در truncated_pages ثبت می‌شوند؛ اگر شاهد مرتبط در بخش حذف‌شده باشد، این محدودیت ممکن است به رتبه آسیب بزند.

نامزدهای فاقد متن با Reranker متنی score نمی‌گیرند و پس از همهٔ نامزدهای امتیازگرفته، با ترتیب شناسه قرار می‌گیرند. این fallback ساده می‌تواند صفحهٔ مرتبط بصریِ بدون متن را عقب بیندازد. راه‌حل‌های ممکن شامل OCR انتخابی، reranking تصویری یا chunk کردن متن و تجمیع score هستند؛ در اجرای فعلی این گزینه‌ها پیاده‌سازی نشده‌اند.

batch_size مرتب‌سازی مجدد پیش‌فرض ۴ است. در اسکریپت مقایسه، پس از تولید رتبه‌های تصویری، مدل تصویری از حافظه آزاد می‌شود و سپس Reranker بارگذاری می‌شود. هدف کاهش هم‌زمانی وزن دو مدل در GPU است. gc.collect و torch.cuda.empty_cache برای آزادسازی منابع استفاده می‌شوند.

۱۹ چرا score به کمک سیگموید درصد نمایش داده شد

در retrieve_fa_hybrid.py برای نمایش تعاملی، score خام Reranker با سیگموید به بازهٔ صفر تا یک نگاشت و در صد ضرب شده است. هدف، خواناترشدن نمایش یک logit بدون دامنهٔ ثابت است. مستندات مدل نیز استفاده از سیگموید برای نگاشت score به این بازه را نشان می‌دهد .

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \qquad score_{\%} = 100\,\sigma(z)$$

```
# scripts_fa/retrieve_fa_hybrid.py
"reranker_score_percent": (
    100.0 * torch.sigmoid(
        torch.tensor(rerank_scores[page_id],
                     dtype=torch.float64)
    ).item()
    if page_id in rerank_scores else None
)
```

score خام z	سیگموید	عدد نمایشی
منفی ۲	۰٫۱۱۹۲	۱۱٫۹۲٪
۰	۰٫۵	۵۰٪
۲	۰٫۸۸۰۸	۸۸٫۰۸٪
۵	۰٫۹۹۳۳	۹۹٫۳۳٪

سیگموید تابعی یکنواخت صعودی است؛ بنابراین اعمال آن ترتیب scoreهای خام را عوض نمی‌کند. رتبه‌بندی با z یا sigmoid از نظر ریاضی یکسان است. برای پرهیز از برخوردهای ناشی از گردکردن درصدهای نزدیک، مرتب‌سازی در کد براساس score خام انجام می‌شود و درصد فقط هنگام نمایش استفاده می‌شود.

$$\sigma'(z) = \sigma(z)(1 - \sigma(z)) > 0$$

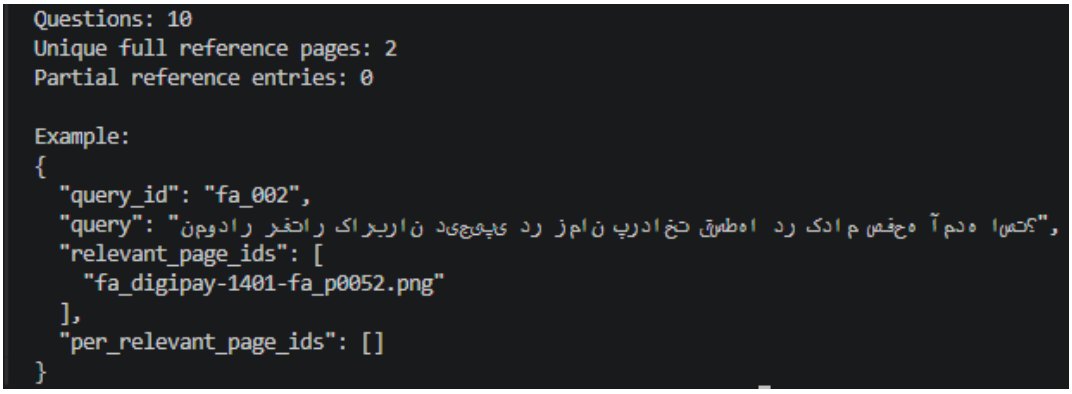
عدد ۹۹٪ به معنای ۹۹٪ احتمال درست‌بودن پاسخ یا مرتبطبودن صفحه نیست. برای چنین تفسیر احتمالاتی، کالیبراسیون روی دادهٔ مستقل لازم است. در این پروژه کالیبراسیون اجرا نشد و بهتر است برچسب خروجی «امتیاز نگاشت‌شده» باشد. درصدهای صفحات هم جمعاً صد نمی‌شوند؛ سیگموید هر زوج را مستقل تبدیل می‌کند و softmax روی نامزدها نیست.

Recall@10 و MRR@10 از شناسه و رتبه محاسبه می‌شوند، نه از درصد سیگموید. فایل مقایسهٔ نهایی score خام Reranker را نگه می‌دارد؛ تبدیل نمایشیِ نسخهٔ تعاملی دلیل بهبود معیارها محسوب نمی‌شود. سیگموید به BM25 score یا MaxSim در ارزیابی نهایی اعمال نشده است.

۲۰ ساخت پرسش و برچسب ارتباط صفحات

نبود qrels آمادهٔ فارسی باعث شد سؤال و صفحهٔ مرجع در data/fa/evaluation/questions_10.json ثبت شوند. هر رکورد query_id، متن query، relevant_page_ids و per_relevant_page_ids دارد. relevant_page_ids صفحات دارای پاسخ یا شواهد کامل است و فیلد دوم برای صفحات تاحدی مرتبط در نظر گرفته شده است.


```
{
  "query_id": "fa_002",
  "query": "نمودار رفتار کاربران دیجی‌پی در زمان پرداخت قسط‌ها در کدام صفحه آمده است؟",
  "relevant_page_ids": [
    "fa_digipay-1401-fa_p0052.png"
  ],
  "per_relevant_page_ids": []
}
```

شکل ۲۱ تعداد سؤال‌ها و صفحات مرجع یکتا و نمونهٔ برچسب fa_002

مجموعهٔ فعلی ۱۰ سؤال، دو صفحهٔ مرجع یکتا و صفر برچسب ارتباط جزئی دارد. همهٔ روش‌ها با همان سؤال‌ها و همان مرجع‌ها ارزیابی شدند. صفحات مرجع تابع خروجی retriever نیستند؛ یک سؤال برای 25BM و 2ColQwen و Hybrid نباید صرفاً به‌دلیل تفاوت رتبه‌بندی، پاسخ مرجع متفاوت بگیرد.

۹ نتیجهٔ دیگر در 10-Top خودکار به per_relevant_page_ids اضافه نمی‌شوند. تعداد صفحات تاحدی مرتبط می‌تواند صفر یا بیشتر باشد و تنها بررسی محتوا آن را تعیین می‌کند. برچسب‌گذاری براساس رتبهٔ خروجی باعث می‌شود همان روش، مرجع ارزیابی خود را تولید کند و مقایسه قابل اعتماد نباشد.

کد ارزیابی یکتایی query_id، غیرخالی‌بودن سؤال و مرجع، نبود تکرار، نبود هم‌پوشانی میان full و partial و حضور تمام مرجع‌ها در corpus را کنترل می‌کند. اگر فیلد partial در ورودی نباشد، مقدار پیش‌فرض آن فهرست خالی است.

۲۱ فرمول Recall و MRR با k برابر ۱۰

R_q مجموعهٔ صفحات کاملاً مرتبط و 10^T_q مجموعهٔ ده نتیجهٔ اول سؤال q است. 10@Recall نسبت صفحات مرجع بازیابی‌شده به همهٔ صفحات مرجع آن سؤال است. میانگین گزارش‌شده از میانگین Recall سؤال‌ها به‌دست می‌آید و در سؤال‌های چندمرجعی، الزاماً با نسبت کلی hitها به همهٔ مرجع‌ها یکسان نیست.

$$Recall@10(q) = \frac{|R_q \cap T_q^{10}|}{|R_q|}$$

$$\overline{Recall@10} = \frac{1}{M} \sum_{q=1}^{M} Recall@10(q)$$

r_q رتبهٔ اولین صفحهٔ مرجع است. اگر r_q حداکثر ۱۰ باشد، 10@RR برابر ۱ تقسیم بر r_q است. اگر مرجع در ده نتیجهٔ اول نباشد، 10@RR صفر است؛ حتی اگر در رتبهٔ ۱۹ پیدا شود. 10@MRR میانگین این مقدار روی M سؤال است. در اجرای فعلی M برابر ۱۰ است.

$$MRR@10 = \frac{1}{M} \sum_{q=1}^{M} RR@10(q)$$

```
57:def metrics(ranking, references):
58-    references = set(references)
59-    if not references:
60-        raise ValueError('Empty references')
61-    ids = ranking[:K]
62-    if len(ids) != len(set(ids)):
63-        raise ValueError('Duplicate IDs in ranking')
64-    ranks = [i for i, p in enumerate(ids, 1) if p in references]
65-    return {
66-        'recall_at_10': len(references & set(ids)) / len(references),
67-        'reciprocal_rank_at_10': 1.0 / min(ranks) if ranks else 0.0,
68-        'first_relevant_rank_at_10': min(ranks) if ranks else None,
69-    }
70-
71-
72-def evaluated_row(question, ranking, details):
73-    full = metrics(ranking, question['relevant_page_ids'])
--
86:def summarize(rows):
87-    count = len(rows)
88-    return {
89-        'questions': count, 'k': K,
90-        'recall_at_10': sum(r['recall_at_10'] for r in rows) / count,
91-        'mrr_at_10': sum(r['reciprocal_rank_at_10'] for r in rows) / count,
92-        'recall_at_10_with_partial': sum(
93-            r['recall_at_10_with_partial'] for r in rows) / count,
94-        'mrr_at_10_with_partial': sum(
95-            r['reciprocal_rank_at_10_with_partial'] for r in rows) / count,
96-    }
97-
98-
99-def load_pages():
100-    path = ROOT / 'data/fa/text/pages_text.jsonl'
101-    pages = {}
102-    for line in path.read_text(encoding='utf-8').splitlines():
```

شکل ۲۲ کد محاسبهٔ معیار با برش [ranking[:K و میانگین روی سؤال‌ها

۲۲ تفسیر معیارها و ارتباط جزئی

با یک مرجع برای هر سؤال، Recall@10 هر سؤال یا صفر است یا یک. به همین دلیل در این آزمایش Recall برابر ۹۰٪ یعنی ۹ سؤال از ۱۰ سؤال hit داشته‌اند. MRR علاوه بر پیداشدن مرجع، نزدیکی آن به ابتدای فهرست را می‌سنجد؛ دو روش با Recall برابر می‌توانند MRR متفاوت داشته باشند.

رتبهٔ اولین مرجع	Recall با یک مرجع	RR@10
۱	۱	۱
۲	۱	۰٫۵
۵	۱	۰٫۲
۹	۱	۰٫۱۱۱۱
۱۹	۰	۰

برای مرجع‌های جزئی، کد معیار مکمل را روی اجتماع صفحات کامل و جزئی محاسبه می‌کند. P_q فهرست برچسب‌های جزئی و A_q مجموعهٔ پذیرفته‌شده است. در این معیار مکمل، هر عضو اجتماع به‌صورت دودویی پذیرفته می‌شود و وزن درجه‌بندی‌شدهٔ جدا ندارد. معیار اصلی همچنان فقط با صفحات کاملاً مرتبط گزارش می‌شود.

$$A_q = R_q \cup P_q, \qquad Recall_{accepted}@10(q) = \frac{|A_q \cap T_q^{10}|}{|A_q|}$$

چون P_q در همهٔ سؤال‌های فعلی خالی است، معیار مکمل با معیار اصلی برابر می‌شود. اگر بعدها partial اضافه شود، Recall مکمل لزوماً بیشتر نمی‌شود؛ مخرج آن هم از |R_q| به |A_q| افزایش پیدا می‌کند. برای ارزیابی درجه‌بندی‌شدهٔ کامل و جزئی می‌توان در مرحلهٔ بعد nDCG را افزود، اما در این مرحله معیار توافق‌شده Recall@10 و MRR@10 بود.

قطعه کد معیار مشترک

```
ids = ranking[:K]
ranks = [i for i, pid in enumerate(ids, 1)
        if pid in references]
recall = len(references & set(ids)) / len(references)
rr = 1.0 / min(ranks) if ranks else 0.0
full = metrics(ranking, question['relevant_page_ids'])
accepted = metrics(
    ranking,
    question['relevant_page_ids'] +
    question['per_relevant_page_ids']
)
```

مراجع فاقد متن از مخرج معیار BM25 حذف نمی‌شوند؛ محدودیت استخراج متن بخشی از عملکرد همان مسیر است. حذف چنین مرجع هایی فقط برای یک روش، مجموعهٔ ارزیابی را متفاوت و مقایسه را خوش‌بینانه می‌کرد.

۲۳ نتایج نهایی مقایسهٔ سه روش

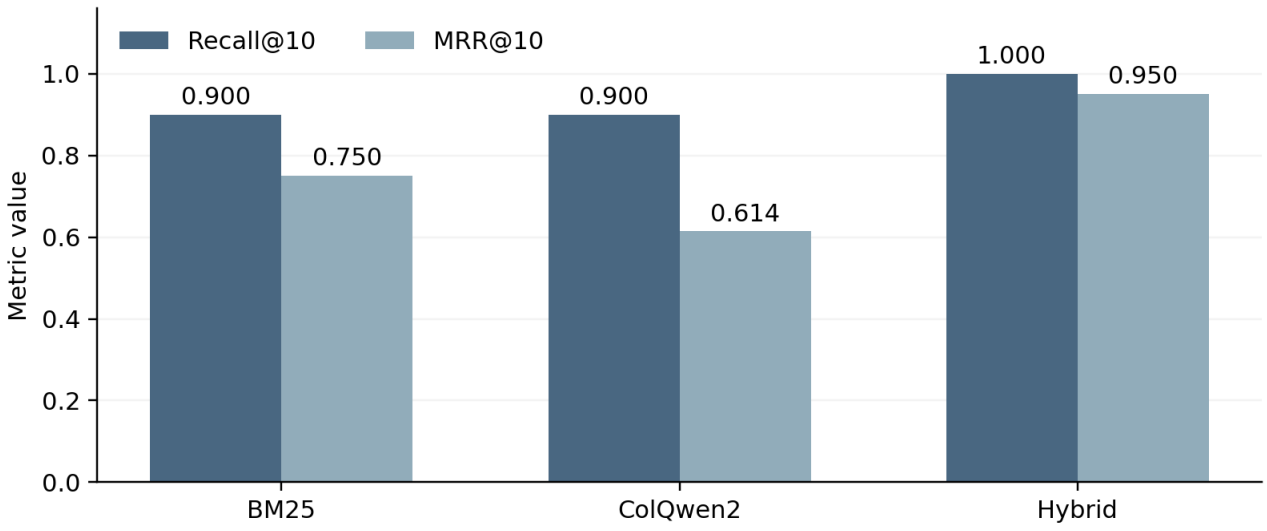
جدول نهایی از اجرای مشترک evaluate_fa_compare.py به‌دست آمد. همان ۱۰ سؤال و همان embedding تصویری ۲۰۴۸ مبنای شاخهٔ 2ColQwen و نامزدسازی تصویری Hybrid بودند. k برای همهٔ روش‌ها برابر ۱۰ بود و برابری score با ترتیب صعودی شناسهٔ صفحه شکسته شد.

روش	Recall@10	MRR@10
BM25	۹۰٪	۰٫۷۵۰۰۰۰
ColQwen2	۹۰٪	۰٫۶۱۴۴۴۴
Hybrid	۱۰۰٪	۰٫۹۵۰۰۰۰

```
Report: comparison_k10_20261013T2035430953962.json

Method       Recall@10       MRR@10
bm25           90.00%     0.750000
colqwen2       90.00%     0.614444
hybrid        100.00%     0.950000
```

شکل ۲۳ خروجی واقعی جدول مقایسهٔ نهایی در ترمینال



نمودار ۱ مقایسهٔ معیارها روی مجموعهٔ ده‌سؤالی فارسی

Hybrid در این مجموعه، نسبت به هر دو روش ۱۰ واحد درصد Recall بیشتر دارد. اختلاف MRR آن با 25BM برابر ۰٫۲۰ و با 2ColQwen حدود ۰٫۳۳۵۶ است. این بهبود هم در پیداشدن مرجعِ سؤال ناموفق و هم در انتقال بیشتر مرجع‌ها به رتبهٔ اول دیده می شود.

25BM و 2ColQwen از نظر Recall برابرند، ولی 25BM در میانگین رتبهٔ اولین مرجع بهتر است. این نتیجه نشان می‌دهد مسیر سادهٔ واژگانی برای همین پرسش‌ها مفید بوده است؛ از آن نمی‌توان برتری عمومی 25BM برای نمودارها یا همهٔ اسناد فارسی را نتیجه گرفت.

۲۴ رتبهٔ مرجع برای تک تک سؤال ها

سؤال	BM25	ColQwen2	Hybrid
fa_001	۱	۵	۱
fa_002	۱۰ بیشتر از	۱۰ بیشتر از	۱
fa_003	۲	۱	۱
fa_004	۲	۱	۱
fa_005	۱	۱	۱
fa_006	۲	۱	۲
fa_007	۱	۲	۱
fa_008	۱	۹	۱
fa_009	۱	۳	۱
fa_010	۱	۱	۱

Query	BM25	ColQwen2	Hybrid
fa_001	1	5	1
fa_002	>10	>10	1
fa_003	2	1	1
fa_004	2	1	1
fa_005	1	1	1
fa_006	2	1	2
fa_007	1	2	1
fa_008	1	9	1
fa_009	1	3	1
fa_010	1	1	1

شکل ۲۴ رتبهٔ اولین صفحهٔ مرجع در خروجی هر روش

در fa_002 هیچ‌یک از دو retriever مستقل مرجع را در ده نتیجهٔ اول نیاوردند، اما Hybrid آن را رتبهٔ اول کرد. رتبهٔ کامل 2ColQwen برای این مرجع ۱۹ بوده است، بنابراین صفحه می‌تواند در نامزدهای تصویری 20-Top وارد Hybrid شود. این توضیح یک مسیر ممکن و سازگار با کد است؛ سهم دقیق شاخهٔ 25BM باید از candidateها در JSON نهایی خوانده شود.

در fa_003 و fa_004، 2 ColQwen مرجع را رتبهٔ اول و 25BM رتبهٔ دوم آورده است. در fa_001 و fa_008، 25 BM رتبهٔ اول دارد، در حالی که 2ColQwen به‌ترتیب رتبهٔ پنجم و نهم دارد. مکمل‌بودن نتایج دو مسیر در این نمونه‌ها دیده می‌شود. Hybrid در fa_006 رتبهٔ دوم گرفته، در حالی که 2ColQwen رتبهٔ اول داشت؛ reranking بهبود تضمین‌شدهٔ هر سؤال نیست.

۲۵ بررسی محاسبات عددی و سؤال ناموفق

برای Hybrid، ۹ سؤال اولین مرجع را در رتبهٔ اول و یک سؤال در رتبهٔ دوم دارد. به همین دلیل 10@MRR برابر ۰٫۹۵ است. 10@Recall برابر یک هم یعنی در همهٔ سؤال‌های تک‌مرجعی، آن مرجع در ده نتیجهٔ اول حضور داشته است.

$$MRR@10_{Hybrid} = \frac{9 \times 1 + 1/2}{10} = 0.95$$

در 25BM، شش سؤال رتبهٔ اول، سه سؤال رتبهٔ دوم و یک سؤال بدون hit در 10-Top هستند. بنابراین MRR برابر (۶ + ۳×۰٫۵) تقسیم بر ۱۰، یعنی ۰٫۷۵ است. این محاسبه با جدول ترمینال و رتبه‌های تک‌سؤالی سازگار است.

$$MRR@10_{ColQwen2} = \frac{1/5 + 0 + 1 + 1 + 1 + 1 + 1/2 + 1/9 + 1/3 + 1}{10} = 0.614444...$$

برای fa_002 در 2ColQwen، صفحهٔ مرجع fa_digipay-1401-fa_p0052.png در رتبهٔ ۱۹ بوده است. بهترین صفحهٔ خروجی 0058p از همان گزارش با score برابر ۲۷٫۴۵۳۱ و سپس 0018p از گزارش ۱۴۰۰ با score برابر ۲۷٫۲۰۳۱ قرار داشتند. نزدیکی scoreهای اولیه صرفاً نزدیک‌بودن امتیاز مدل را نشان می‌دهد؛ برچسب مرتبطبودن باید با محتوای صفحه تعیین شود.

در گزارش قدیمی که RR خارج از 10-Top را هم حساب می‌کرد، رتبهٔ ۱۹ سهم ۱۹/۱ داشت. با معیار فعلی 10@MRR این سهم صفر است. اختلاف MRR نامحدودِ ۰٫۶۱۹۷۰۸ با 10@MRR ۰٫۶۱۴۴۴۴، برابر ۱ تقسیم بر (۱۰×۱۹) است. بنابراین تغییر این دو عدد از تفاوت تعریف معیار می‌آید و به معنی تغییر نتیجهٔ مدل نیست.

برای این سؤال، صحت صفحهٔ مرجع، یکتایی شناسه و تعداد صفحات بررسی شد. manifest نیز بودجهٔ تصویری و شکل embeddingها را ثبت کرده است. این شواهد سلامت ساختار اجرایی را پشتیبانی می‌کنند، اما به‌تنهایی نشان نمی‌دهند چرا مدل صفحهٔ درست را پایین‌تر رتبه‌بندی کرده است. تحلیل زبان، وضوح نمودار و حساسیت پردازشگر به آزمایش جدا نیاز دارد.

۲۶ چالش های مسیرها و ساختار داده و فایل ها

ساختار واقعی dataset و تفاوت نام فایل ها

پیش از اجرای ابزارها باید معلوم می‌شد دادهٔ واقعی شامل PDF، تصویر، Parquet یا metadata است و هر اسکریپت چه schemaای می‌خواهد. راه‌حل، بررسی مستقیم تعداد صفحات، schema Parquet و نمونهٔ pages.jsonl بود. شناسهٔ corpus به‌عنوان کلید مشترک میان تصویر، embedding و مرجع سؤال انتخاب شد. تکیه بر نام حدسی فایل به جای بررسی وجود و schema، خطاهای مسیر را تکرار می‌کرد.

پوشهٔ مخزن و پوشهٔ داخلی پروژه

وجود دو سطح FinRAGBench-V-git و FinRAGBench-V باعث شد مسیر محیط مجازی و برخی فایل‌ها اشتباه فرض شوند. دستور source .venv/bin/activate در پوشهٔ داخلی شکست خورد؛ در همان زمان محیط اصلی فعال بود. راه‌حل، تثبیت پوشهٔ اجرا، بررسی Python و استفاده از ROOT مشتق‌شده از مسیر خود اسکریپت بود. داده‌ها از ابتدا دوباره unzip یا آماده‌سازی نشدند، چون خروجی معتبر موجود بود.

فایل ناقص اسکریپت مقایسه

اولین نسخهٔ کپی‌شدهٔ evaluate_fa_compare.py فقط ۳۵۸۲ بایت داشت و با تعریف summarize تمام می‌شد.
py_compile موفق بود، چون فایل از نظر syntax معتبر بود، اما مسیر main و فراخوانی اجرا در آن نبود؛ در نتیجه هیچ پوشهٔ fa_comparison و خروجی نهایی ساخته نمی‌شد. این رخداد نشان داد موفقیت بررسی syntax به معنی کامل‌بودن برنامه نیست.

با wc -c، head و tail ناقص‌بودن فایل مشخص شد. راه‌حل، انتقال یکجای نسخهٔ کامل، بررسی وجود main و سپس اجرای واقعی بود. پس از اصلاح، جدول سه روش و هر سه فایل JSON و CSV ایجاد شدند. برای جلوگیری از تکرار، نسخهٔ کامل فایل و بررسی اندازه و hash بهتر از کپی تکه‌تکهٔ متن بلند است.

نبود مرجع آماده و مدیریت subset

برای فارسی qrels آماده نداشتیم؛ بنابراین مجموعهٔ کوچک سؤال و مرجع دستی ساختیم و metadata همهٔ ۱۲۲ صفحه را نگه داشتیم. subset ارزیابی از corpus مستقل است: سؤال‌ها فقط دو صفحهٔ مرجع دارند، اما جست‌وجو روی corpus کامل انجام می‌شود. حذف صفحات غیرمرجع از corpus، مسئله را مصنوعی آسان می‌کرد و در اجرای نهایی انجام نشد.

۲۷ چالش های حافظه و cache و کیفیت فارسی

حافظهٔ GPU و تنظیم بودجهٔ تصویر

در مسیر توسعه، افزایش بودجهٔ تصویر تا ۳۰۷۲ با خطای کمبود حافظهٔ GPU روبه‌رو شد و اجرای کاملِ قابل مقایسه نداد. خروجی نهایی روی ۲۰۴۸ تثبیت شد. افزایش تعداد بردارهای تصویر هزینهٔ encoding و score را بالا می‌برد، اما لزوماً کیفیت بهتر را تضمین نمی کند؛ در نتیجه مقدار نهایی با سلامت اجرای کامل و نتیجهٔ توسعه انتخاب شد.

راه‌حل‌های اجرایی شامل batch_size=1 هنگام encoding، float16، SDPA، checkpoint صفحه‌ای و جداکردن زمان حضور مدل تصویری و Reranker در GPU بود. مصرف اوج حافظه و latency مستقل هر روش در جدول نهایی اندازه‌گیری نشده‌اند؛ از این تنظیمات نباید عدد دقیق صرفه‌جویی ساخت.

cache مدل و حجم دیسک

وزن‌های 2ColQwen محلی در retrieval/colqwen2-v1.0-merged/ و cache مدل Reranker در models/hf-cache/ نگهداری شدند. local_files_only=True در بارگذاری Reranker از دانلود ناخواسته هنگام ارزیابی جلوگیری می‌کند و manifest امکان تثبیت revision را می‌دهد. cache قدیمی و چند نسخهٔ embedding می‌توانند فضای دیسک را مصرف کنند؛ حذف آن‌ها باید پس از تشخیص وابستگی فایل‌ها انجام شود.

در پاک‌سازی، فایل‌های .bak غیرضروری حذف شدند و خروجی‌های تعاملی قدیمی و اسکریپت آزمایشی بیرون مخزن به آرشیو محلی منتقل شدند. داده‌های اصلی، مدل‌ها، index و embedding منتخب حفظ شدند. نگه‌داشتن JSON و CSV مقایسهٔ نهایی، امکان پیگیری عددها را پس از پاک‌سازی فراهم کرد.

تفاوت نتیجهٔ فارسی و نقش پردازشگر

تفاوت فارسی و انگلیسی می‌تواند در tokenization سؤال، خواندن نوشتهٔ داخل تصویر و نگاشت متن و نمودار اثر داشته باشد. در این پروژه مقایسهٔ کنترل‌شدهٔ همان سؤال به فارسی و انگلیسی انجام نشده است؛ بنابراین خطای fa_002 را قطعی به زبان فارسی نسبت نمی دهیم. کنترل processor، مسیر مدل، بودجهٔ تصویر و مرجع صفحه، پیش‌نیاز چنین تحلیل زبانی است.

آشفتگی متن در برخی خروجی‌های ترمینال نیز صرفاً از تصویر خروجی قابل ریشه‌یابی نیست؛ نمایش bidi و ترتیب استخراج هر دو باید بررسی شوند. sort=False و نرمال‌سازی مشکل تطبیق حروف را کاهش می‌دهند، اما بازسازی ساختار خواندن فارسی یا OCR را جایگزین نمی‌کنند.

۲۸ چرا Vector DB و OCR استفاده نشد

ذخیرهٔ فایل و MaxSim مستقیم

برای corpus کوچکِ ۱۲۲صفحه‌ای، نگه‌داشتن embedding در فایل و امتیازدهی مستقیم، راه‌اندازی و بررسی را ساده کرد. شناسه ها کنار بردارها نگه داشته می‌شوند و نتیجه به metadata وصل می‌شود. استفاده از Vector DB نیازمند سرویس، schema، مدیریت index و انتخاب روش مناسب برای embedding چندبرداری بود؛ در این اندازهٔ آزمایش، آن پیچیدگی برای رسیدن به هدف اولیه لازم نبود.

Vector DB گزینهٔ نامناسبی برای همهٔ پروژه‌ها نیست. بعضی سامانه‌ها از multi-vector و late interaction پشتیبانی می‌کنند و در corpus بزرگ می‌توانند برای مدیریت، فیلتر و جست‌وجوی سریع‌تر مفید باشند. جایگزین‌کردن MaxSim با یک بردار میانگین شده هم تغییر الگوریتم است، نه فقط تغییر محل ذخیره. پیش از مهاجرت باید معیار و هزینهٔ جست‌وجو با همان داده اندازه‌گیری شود.

عدم اجرای OCR در نسخهٔ فعلی

در مسیر 2ColQwen، تصویر صفحه مستقیماً ورودی مدل است و برای ساخت embedding به متن OCR وابستگی نداریم. در مسیر 25BM نیز لایهٔ متنی موجود در PDF برای ۱۱۲ صفحه توکن قابل جست‌وجو فراهم کرد؛ بنابراین ابتدا استخراج مستقیم اجرا شد. این انتخاب هزینهٔ توسعه و پردازش OCR را کم کرد و از واردکردن خطای یک موتور OCR آزمایش‌نشده به خط مبنا جلوگیری کرد.

این تصمیم محدودیت دارد: ۱۰ صفحه در index متنی حضور ندارند و متن استخراج‌شده هم ممکن است ترتیب ناقص داشته باشد. OCR انتخابی برای صفحه‌های فاقد متن یا با استخراج خراب، گزینهٔ منطقی مرحلهٔ بعد است. کیفیت فارسی، جدول، عدد و ترتیب خواندن باید روی نمونهٔ برچسب‌خورده سنجیده شود؛ ادعا نمی‌کنیم OCR الزاماً دقت کمتری دارد، چون در این پروژه مقایسه نشده است.

گزینه های دیگر برای ترکیب نتایج

جمع وزنی scoreهای 25BM و MaxSim، به هم‌مقیاس‌سازی و تنظیم وزن نیاز دارد. ترکیب رتبه با RRF می‌تواند گزینه‌ای ساده‌تر و بدون Reranker باشد. در نسخهٔ فعلی، ادغام نامزد و Reranker انتخاب شد تا یک مدل مشترک، ارتباط زوج سؤال و صفحه را بسنجد. این انتخاب نسبت به 25BM هزینهٔ مدل بیشتری دارد و به کیفیت متن وابسته است؛ مقایسهٔ RRF و fusion وزن‌دار در این مرحله انجام نشده است.

۲۹ دستورهای اجرای مقایسه و کنترل نتیجه

دستور زیر همان مسیر مقایسهٔ نهایی را اجرا می‌کند. محیط Python اصلی پروژه باید فعال باشد. مدل‌ها، index و embedding از قبل موجودند؛ اجرای این دستور corpus را دوباره encode نمی‌کند. اجرای مجدد یک خروجی با timestamp جدید می‌سازد و برای ثبت جدول موجود، کافی است JSON نهایی خوانده شود.

```
cd /home/baseri/FinRAGBench-V-git/FinRAGBench-V
python -m py_compile scripts_fa/evaluate_fa_compare.py
python scripts_fa/evaluate_fa_compare.py \
  --questions data/fa/evaluation/questions_10.json \
  --embedding-dir retrieval/encode/colqwen_fa_2048 \
  --methods colqwen2 bm25 hybrid
```

خواندن جدول اجرای ثبت شده

```
python - <<'PY'
import json
from pathlib import Path
path = Path(
    "retrieval/results/fa_comparison/"
    "comparison_k10_20261013T203543095396Z.json"
)
report = json.loads(path.read_text(encoding="utf-8"))
for method in ["bm25", "colqwen2", "hybrid"]:
    s = report["methods"][method]["summary"]
    print(method, s["recall_at_10"], s["mrr_at_10"])
PY
```

کنترل فایل اجرایی و محیط

```
pwd
command -v python
python --version
wc -c scripts_fa/evaluate_fa_compare.py
tail -n 8 scripts_fa/evaluate_fa_compare.py
ls -lh retrieval/encode/colqwen_fa_2048/
ls -lh retrieval/results/fa_comparison/
git status -sb
```

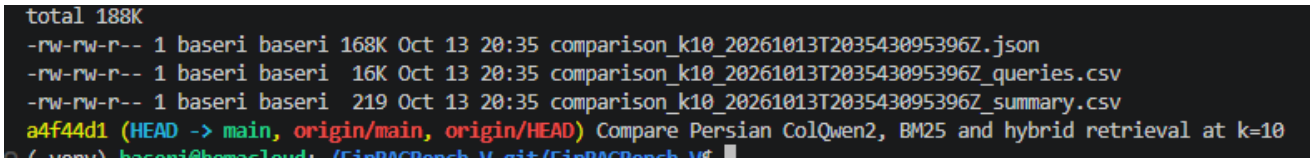
اجرای مستقیم مسیر فایل Python در shell با اجرای python file.py یکسان نیست. برای جلوگیری از خطای permission یا نبود shebang، اسکریپت با Python محیط پروژه فراخوانی می‌شود. اگر فایل وجود ندارد، ابتدا مسیر و نام را بررسی می‌کنیم؛ داده و مدل از ابتدا دوباره ساخته نمی‌شوند.

۳۰ فایل های خروجی و ثبت نسخهٔ نهایی

سه فایل نهایی در /retrieval/results/fa_comparison ذخیره شده‌اند. پیشوند مشترک زیر، اجرای ثبت‌شدهٔ مقایسه را مشخص می‌کند. زمان انتهای نام در قالب UTC است و شناسهٔ خروجی، مدل یا بودجهٔ embedding را تغییر نمی‌دهد.

comparison_k10_20261013T203543095396Z

پسوند فایل	محتوا
.json	نتایج کامل سؤال‌ها و 10-Top و summary و provenance
_summary.csv	جدول تجمیعی هر روش و معیارهای کامل و مکمل
_queries.csv	نتیجهٔ هر سؤال و رتبه و معیار و شناسه‌های بازیابی



شکل ۲۵ وجود هر سه خروجی و ثبت کامیت مقایسه روی main و origin/main

```
git log -1 --oneline
# a4f44d1 Compare Persian ColQwen2, BM25 and hybrid retrieval at k=10
```

در JSON، مسیر dataset و 256SHA آن، 256SHA اسکریپت، شیوهٔ tie-breaking، تعداد مرجع‌های یکتا، نسخهٔ بسته‌ها، manifest embedding و اطلاعات index و Reranker ثبت می‌شوند. این اطلاعات برای پیگیری تغییرها مهم‌اند؛ نتیجه بدون اشاره به سؤال‌ها و تنظیمات اجرا، قابل تفسیر دقیق نیست.

total_batch_seconds زمان کل اجرای دسته‌ای است. چون رتبه‌های تصویری بین 2ColQwen و Hybrid استفادهٔ مشترک دارند، این عدد latency مستقل یک روش نیست و نباید برای نتیجه‌گیری دربارهٔ سرعت سه روش به‌کار رود. مقایسهٔ زمان نیازمند warmup، شرایط GPU ثابت و اندازه‌گیری جداگانه است.

پاک سازی خروجی های قدیمی

فایل‌های backup حذف شدند و خروجی‌های قدیمی fa و 25fa_bm و fa_hybrid و گزارش ارزیابی تصویری قبلی به آرشیو بیرون مخزن منتقل شدند. Archived به معنی انتقال برای نگهداری بود، نه حذف داده. فایل اتفاقی ناشی از دستور Git نیز حذف شد. خروجی نهایی سه روش و فایل‌های لازم برای اجرای پروژه باقی ماندند.

/home/baseri/FinRAGBench-V-local-archive/
20261013T213224201492Z/

۳۱ جمع بندی مهندسی و حدود نتیجه گیری

خروجی این مرحله یک زنجیرهٔ اجرایی از PDF فارسی تا صفحات Top-10 است. corpus تصویری ۱۲۲ صفحه دارد، 25BM روی ۱۱۲ صفحهٔ دارای توکن اجرا می‌شود، embedding تصویری با بودجهٔ ۲۰۴۸ ذخیره شده و Hybrid با ادغام نامزد و Reranker نتایج را مرتب می‌کند. کنترل شناسه و schema و hash و checkpoint، بخش مهم قابل تکرارشدن اجرای محلی بود.

نتیجهٔ اصلی این آزمایش، موفقیت Hybrid در همهٔ ۱۰ سؤال و انتقال ۹ مرجع به رتبهٔ اول است. همزمان، fa_006 نشان می‌دهد reranking همیشه از retriever تصویری بهتر نیست. نگه‌داشتن رتبه‌های تک‌سؤالی اجازه می‌دهد به‌جای اتکا به یک عدد تجمیعی، بهبود و افت هر سؤال بررسی شود.

محدودیت های مجموعهٔ فعلی

ده سؤال فقط دو صفحهٔ مرجع یکتا دارند و پوشش محتوایی آن‌ها محدود است. همین مجموعه در توسعه و انتخاب تنظیمات استفاده شده، بنابراین آزمون نهاییِ مستقل محسوب نمی‌شود. برچسب ارتباط جزئی هنوز وارد نشده و صفر بودن آن به معنای نبود هیچ صفحهٔ تاحدی مرتبط در corpus نیست. همچنین OCR، تغییر DPI،fusion های دیگر و کالیبراسیون درصد Reranker آزمایش نشده‌اند.

نتایج حاضر برای اثبات کارکرد مسیر و مقایسهٔ اولیه مفیدند. برای ادعای برتری عمومی در فارسی باید سؤال‌ها روی صفحات بیشتر، انواع متن و جدول و نمودار و چند سند متنوع‌تر توزیع شوند. لازم است مجموعهٔ آزمون مستقل پیش از تنظیم پارامترها کنار گذاشته شود و با همان برچسب‌ها و تعریف معیار برای همهٔ روش‌ها ارزیابی شود.

گام های بعدی

اولویت بعدی، گسترش و بازبینی برچسب‌ها و تحلیل خطاهای بازیابی است. پس از آن می‌توان OCR انتخابی، chunking متن برای Reranker و مقایسهٔ روش ترکیب رتبه را بررسی کرد. اندازه‌گیری زمان و حافظهٔ هر روش باید جدا انجام شود. اتصال به API تولید پاسخ، پس از تثبیت مسیر بازیابی و با استفاده از تصویر و مسیر صفحات منتخب انجام خواهد شد.

در بخش تولید، پاسخ باید به شواهد صفحهٔ بازیابی‌شده متصل شود؛ معیار بازیابیِ خوب به‌تنهایی صحت پاسخ نهایی را تضمین نمی‌کند. ارزیابی پاسخ و استناد بصری، مرحله‌ای مستقل از Recall@10 و MRR@10 فعلی است.

۳۲ منابع و راهنمای شواهد

منابع علمی و مستندات فنی زیر برای تعریف روش‌ها استفاده شده‌اند. نتایج فارسی از خروجی اجرای محلی و اسکرین‌های پروژه به‌دست آمده‌اند و اعداد بنچمارک اصلی به‌عنوان نتیجهٔ آزمایش فارسی گزارش نشده‌اند.

۱ Zhao, S., Jin, Z., Li, S., Gao, J. . FinRAGBench-V A Benchmark for Multimodal RAG with Visual Citation in the Financial Domain. مقالهٔ پیوست، چکیده و بخش معرفی corpus.

https://huggingface.co/datasets/zhaosuifeng/FinRAGBench-V
https://github.com/zhaosuifeng/FinRAGBench-V

۲ Dorian Brown. rank_bm25. پیاده‌سازی BM25Okapi، مقدار پیش‌فرض پارامترها و قاعدهٔ IDF منفی. مستند فنی برای تطبیق فرمول با کتابخانه.

https://github.com/dorianbrown/rank_bm25/blob/master/rank_bm25.py

۳ Faysse et al. ColPali Efficient Document Retrieval with Vision Language Models .. تعریف بازیابی چندبرداری و تعامل دیرهنگام. فرمول MaxSim برای توضیح منطق امتیاز استفاده شده است؛ کد اجرا، 2ColQwen است.

https://arxiv.org/abs/2407.01449

۴ BAAI. bge-reranker-v2-m3 model card .. توضیح score زوج سؤال و متن و نگاشت با sigmoid. نسخهٔ اجرای پروژه از reranker_manifest.json خوانده می‌شود.

https://huggingface.co/BAAI/bge-reranker-v2-m3